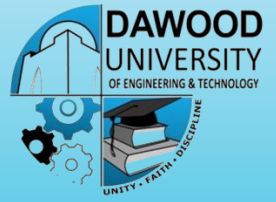
LECTURE – CLASS AND OBJECT
OOP (JAVA)

MR. PREM KUMAR
LECTURER
DEPARTMENT OF CYBER SECURITY

Dawood University of Engineering and Technology, Karachi

AGENDA



- OOP Introduction
- Programming
- Class
- Object
- Method

What is OOP?

- The term used to describe a programming approach based on **objects** and **classes**.
- A paradigm allows us to organize software as a collection of objects that consist of both data and behavior.
- In 1980s the word 'object' has appeared in relation to programming languages.
- Since 1990 having object-oriented features.
- It is widely accepted that object-oriented programming is the most important and powerful way of creating software.

Programming

- Procedural programming is about writing procedures or functions that perform operations on the data.

  FORTRAN, ALGOL, COBOL, BASIC, Pascal and C.

- Object Oirented Programming is about creating objects that contain both data and functions.

  Java, C++, C#, Python, PHP, JavaScript, Ruby, Perl

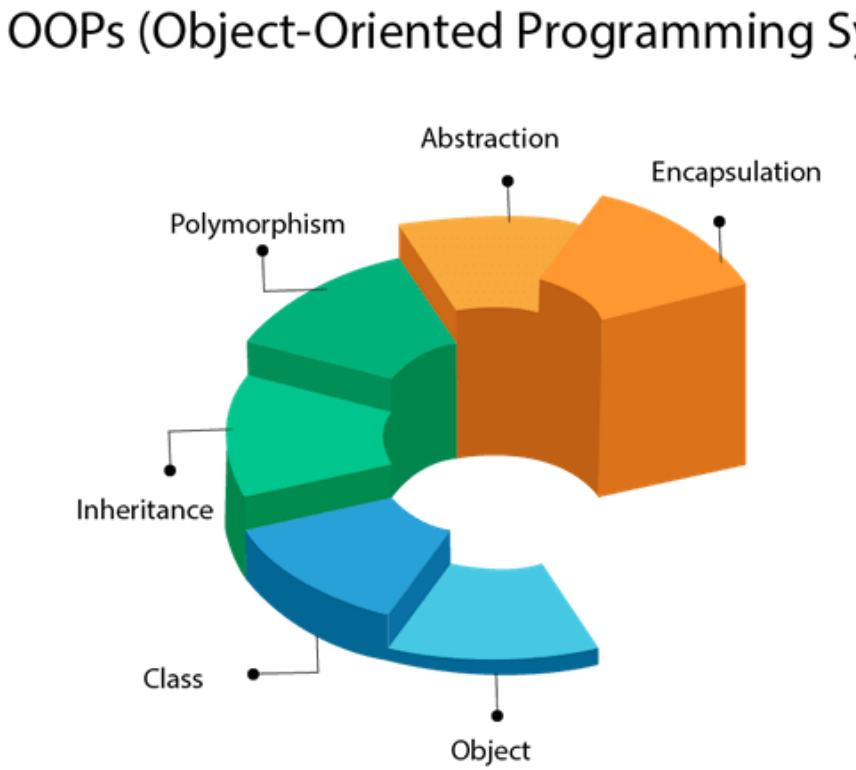
- **Simula** is considered the first object-oriented programming language. The programming paradigm where everything is represented as an object is known as a truly object-oriented programming language.
- **Smalltalk** is considered the first truly object-oriented programming language.
- The popular object-oriented languages are Java, C#, PHP, Python, C++, etc.
- The main aim of object-oriented programming is to implement real-world entities, for example, object, classes, abstraction, inheritance, polymorphism, etc.

DIFFERENCE

Procedural Oriented Programming	Object Oriented Programming
program is divided into small parts called ***functions***	program is divided into small parts called ***objects***.
follows ***top down approach***.	follows ***bottom up approach***.
no access specifier in procedural programming.	access specifiers like private, public, protected etc.
Adding new data and function is not easy.	Adding new data and function is easy.
does not have any proper way for hiding data so it is ***less secure***.	provides data hiding so it is ***more secure***.
function is more important than data.	data is more important than function.
based on ***unreal world***.	based on ***real world***.
Examples: C, FORTRAN, Pascal, Basic etc.	Examples: C++, Java, Python, C# etc.

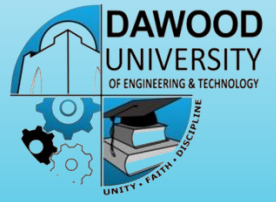
- **Object** means a real-world entity such as a pen, chair, table, computer, watch, etc. **Object-Oriented Programming** is a methodology or paradigm to design a program using classes and objects. It simplifies software development and maintenance by providing some concepts:

Object Oriented Programming



- Modularization: where the application can be decomposed into modules.
- Software re-use: where an application can be composed from existing and new modules.
- generally, supports five main features:
  1. Classes
  2. Objects
  3. Polymorphism
  4. Inheritance
  5. Abstraction
  6. Encapsulation

Class

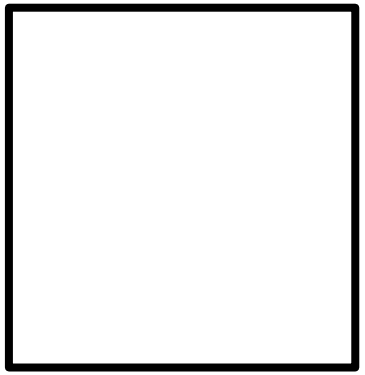


- *Collection of objects* is called class. It is a logical entity.
- A class can also be defined as a blueprint from which you can create an individual object. Class doesn't consume any space.

- Defines object properties including a valid range of values, and a default value.
- Provide a well-defined interface
- Represent a clear concept
- describes object behavior
- Example: Box, Car, Television,..

Class

- A class is a template for objects.
- defines object properties including a valid range of values, and a default value.
- describes object behavior.
- Example: Box, Car, Television,



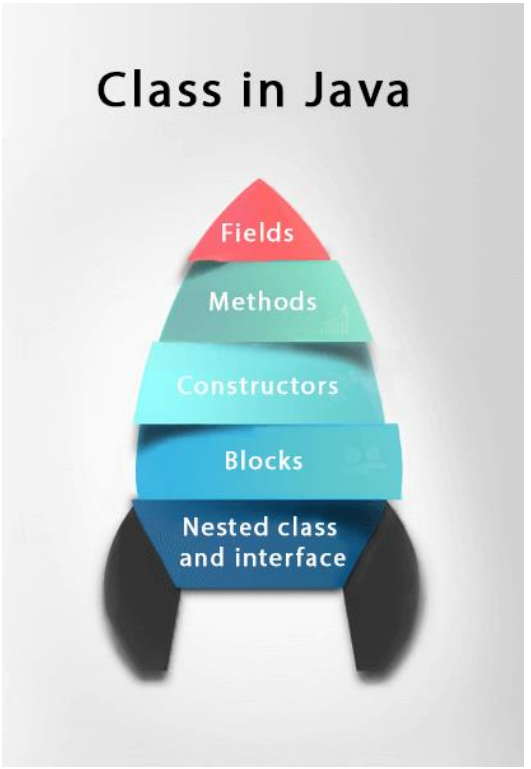
Box

Car

Television

Class

- A class is a group of objects which have common properties. It is a template or blueprint from which objects are created. It is a logical entity. It can't be physical.
- A class in Java can contain:
  - **Fields**
  - **Methods**
  - **Constructors**
  - **Blocks**
  - **Nested class and interface**



Class

Create a Class

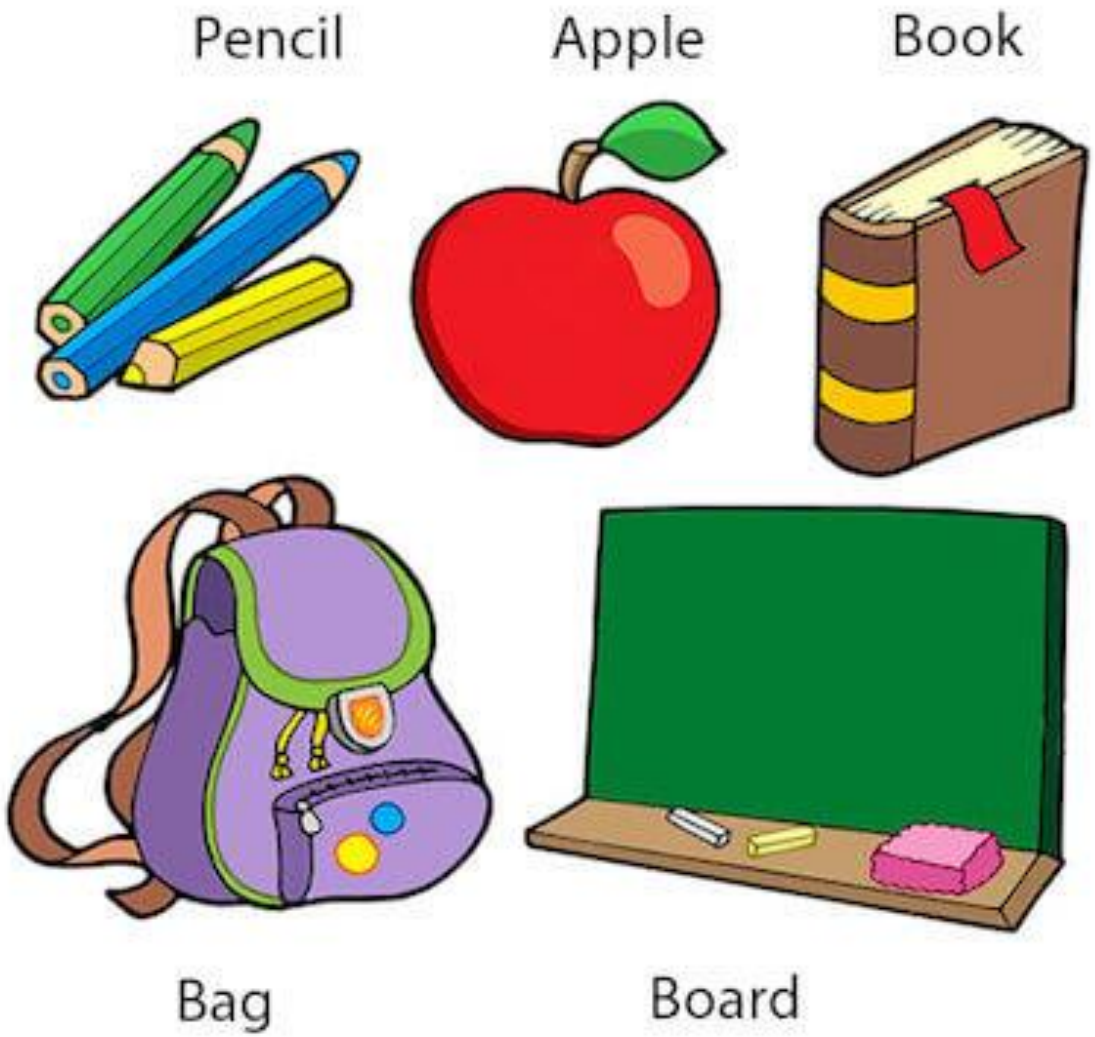
- To create a class, use the keyword **`class`**?

```
public class Main {
  int x = 5;
}
```

OBJECT

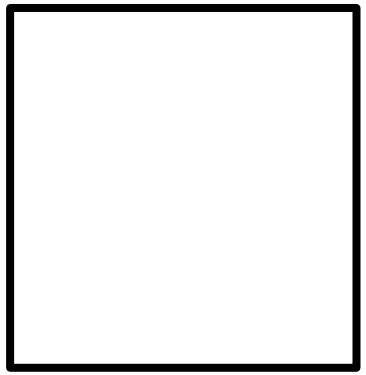
- Any entity that has state and behavior is known as an object. For example, a chair, pen, table, keyboard, bike, etc. It can be physical or logical.
- An Object can be defined as an instance of a class. An object contains an address and takes up some space in memory. Objects can communicate without knowing the details of each other's data or code. The only necessary thing is the type of message accepted and the type of response returned by the objects.
- **Example:** A dog is an object because it has states like color, name, breed, etc. as well as behaviors like wagging the tail, barking, eating, etc.

Objects: Real World Examples



An Object

- An **object** is an instance of a class / is created from a class.



Box

Car

Television

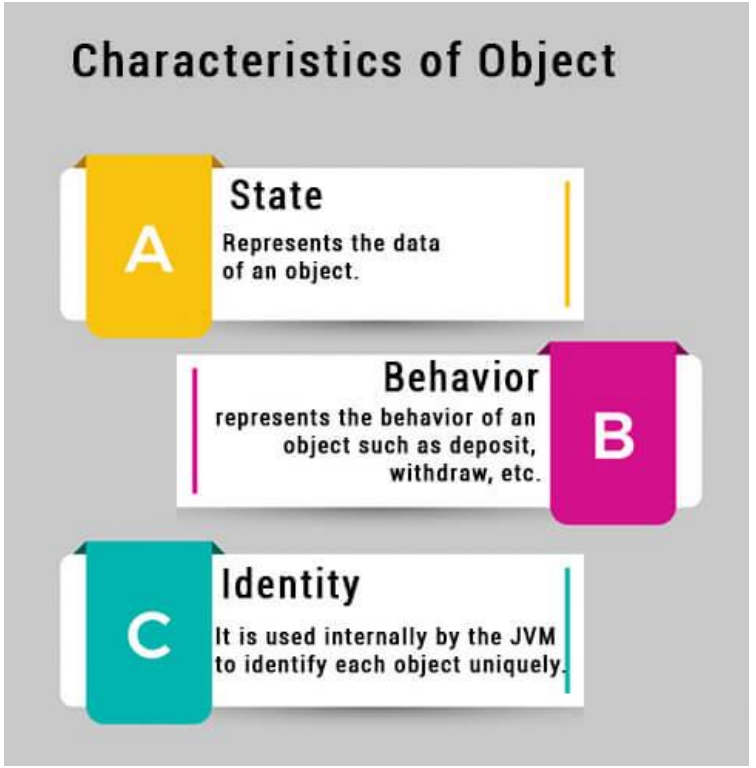
- Example: a dog.

  breed, color, …

  behavior is - barking, wagging the tail, running.

CHARACTERISTICS OF OBJECT

- **State:** represents the data (value) of an object.
- **Behavior:** represents the behavior (functionality) of an object such as deposit, withdraw, etc.
- **Identity:** An object identity is typically implemented via a unique ID. The value of the ID is not visible to the external user. However, it is used internally by the JVM to identify each object uniquely.
- For Example, Pen is an object. Its name is Reynolds; color is white, known as its state. It is used to write, so writing is its behavior.



An Object

- If you compare the software object with a real-world object, they have very similar characteristics.
- Software objects also have a state and a behavior. A software object's state is stored in fields and behavior is shown via methods.

Create an Object

- An object is created from a class.
- In Java, the new keyword is used to create new objects.

There are three steps when creating an object from a class

- **Declaration** − A variable declaration with a variable name with an object type.
- **Instantiation** − The 'new' keyword is used to create the object.
- **Initialization** − The 'new' keyword is followed by a call to a constructor. This call initializes the new object.

SUMMARIZE THE DEFINITION OF OBJECT

- An object is *a real-world entity*.
- An object is *a runtime entity*.
- The object is *an entity which has state and behavior*.
- The object is *an instance of a class*.

An Object

Create an Object

```
1  public class Puppy {
2     public Puppy(String name) {
3        // This constructor has one parameter, name.
4        System.out.println("Passed Name is :" + name );
5     }
6
7     public static void main(String []args) {
8        // Following statement would create an object myPuppy
9        Puppy myPuppy = new Puppy( "tommy" );
10    }
11 }
```

Result:

```
Passed Name is :tommy
```

An Object

Accessing Instance Variables and Methods

- Instance variables and methods are accessed via created objects. To access an instance variable, following is the fully qualified path:

```
/* First create an object */
ObjectReference = new Constructor();
/* Now call a variable as follows */
ObjectReference.variableName;
/* Now you can call a class method as follows */
ObjectReference.MethodName();
```

Java Methods

- A **method** is a block of code which only runs when it is called.
- You can pass data, known as parameters, into a method.
- Methods are used to perform certain actions, and they are also known as **functions**.
- Why use methods? To reuse code: define the code once, and use it many times.

Create a method

- A method must be declared within a class.
- It is defined with the name of the method, followed by parentheses **()**.
- Java provides some pre-defined methods, such as System.out.println(), but you can also create your own methods to perform certain actions:

Example:

```
public class Main {
  static void myMethod() {
    // code to be executed
  }
}
```

Create a method

Example:

```
public class Main {
  static void myMethod() {
    // code to be executed
  }
}
```

- myMethod() is the name of the method.
- static means that the method belongs to the Main class and not an object of the Main class.
- Will learn more about objects and how to access methods through objects later.
- void means that this method does not have a return value. Will learn more about return values later.

Call a method

- To call a method in Java, write the method's name followed by two parentheses **()** and a semicolon**;**
- In the following example, myMethod() is used to print a text (the action), when it is called:

Example:

Inside main, call the myMethod() method:

```
static void myMethod() {
  System.out.println("I just got executed!");
}

public static void main(String[] args) {
  myMethod();
```

Call a method

Inside main, call the myMethod() method:

Example:

```
static void myMethod() {
  System.out.println("I just got executed!");
}

public static void main(String[] args) {
  myMethod();
}
```

```
public class Main {
  static void myMethod() {
    System.out.println("I just got executed!");
  }

  public static void main(String[] args) {
```

Result:

```
I just got executed!
```

Call a method

A method can also be called multiple times:

Example:

```
public class Main {
  static void myMethod() {
    System.out.println("I just got executed!");
  }

  public static void main(String[] args) {
    myMethod();
    myMethod();
    myMethod();
  }
}
```

Result:

```
I just got executed!
I just got executed!
I just got executed!
```

Create a method

- Considering the following example to explain the syntax of a method.

Syntax

```
public static int methodName(int a, int b) {
   // body
}
```

- **public static** − modifier
- **int** − return type
- **methodName** − name of the method
- **a, b** − formal parameters
- **int a, int b** − list of parameters

Create a method

- **modifier** − It defines the access type of the method and it is optional to use.
- **returnType** − Method may return a value.
- **nameOfMethod** − This is the method name. The method signature consists of the method name and the parameter list.
- **Parameter List** − The list of parameters, it is the type, order, and number of parameters of a method. These are optional, method may contain zero parameters.
- **method body** − The method body defines what the method does with the statements.

Create a method

- This method takes two parameters num1 and num2 and returns the maximum between the two.

```
public static int minFunction(int n1, int n2) {
   int min;
   if (n1 > n2)
      min = n2;
   else
      min = n1;

   return min;
}
```

Call a method

Example:

```
public class ExampleMinNumber {

   public static void main(String[] args) {
      int a = 11;
      int b = 6;
      int c = minFunction(a, b);
      System.out.println("Minimum Value = " + c);
   }

   /** returns the minimum of two numbers */
   public static int minFunction(int n1, int n2) {
      int min;
      if (n1 > n2)
         min = n2;
      else
         min = n1;

      return min;
   }
```

Result:

```
Minimum value = 6
```

Thanks ☺